Cursul 4

Grafică pe calculator

2025

Dumnezeu a zis: *Să fie lumină!* **Și a fost lumină.**

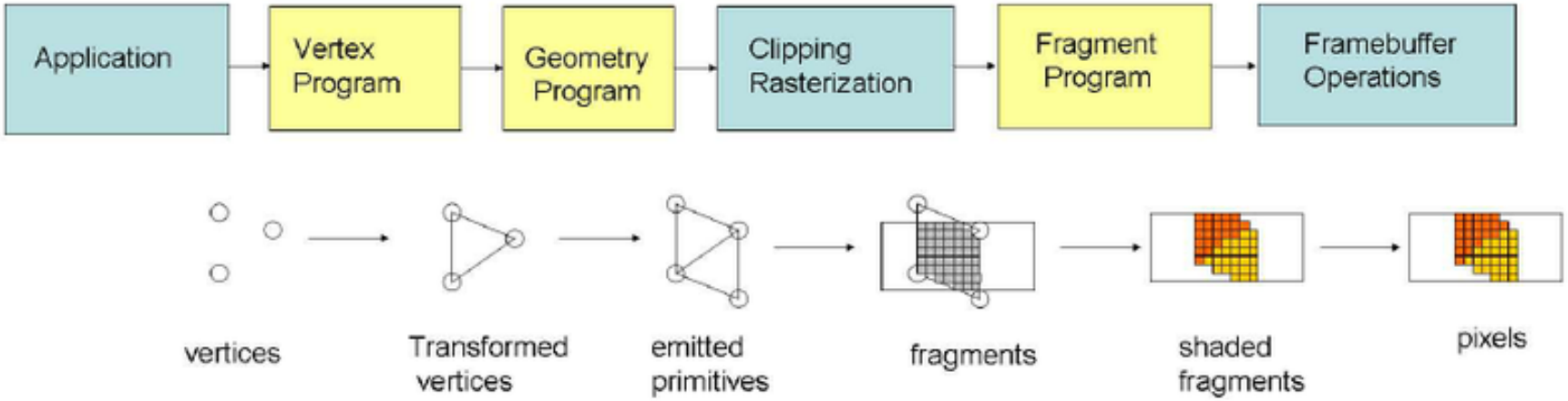
Geneza 1:3

Cuprins

1. OpenGL pipeline
2. Puțină fizică nu strică
   - Reflexia luminii
   - Refracția
   - Culoarea luminii
3. Modelarea Luminii
   - Modele locale
     - Modelul Phong
   - Modele globale
4. Materiale și Lumină
   - Reflecția luminii
   - Strălucirea (luciul)
   - Ecuația Luminii
   - Componentele culorii materialului
   - Exemple de materiale
   - Exemple de cod

OpenGL pipeline

Shadere



Shadere

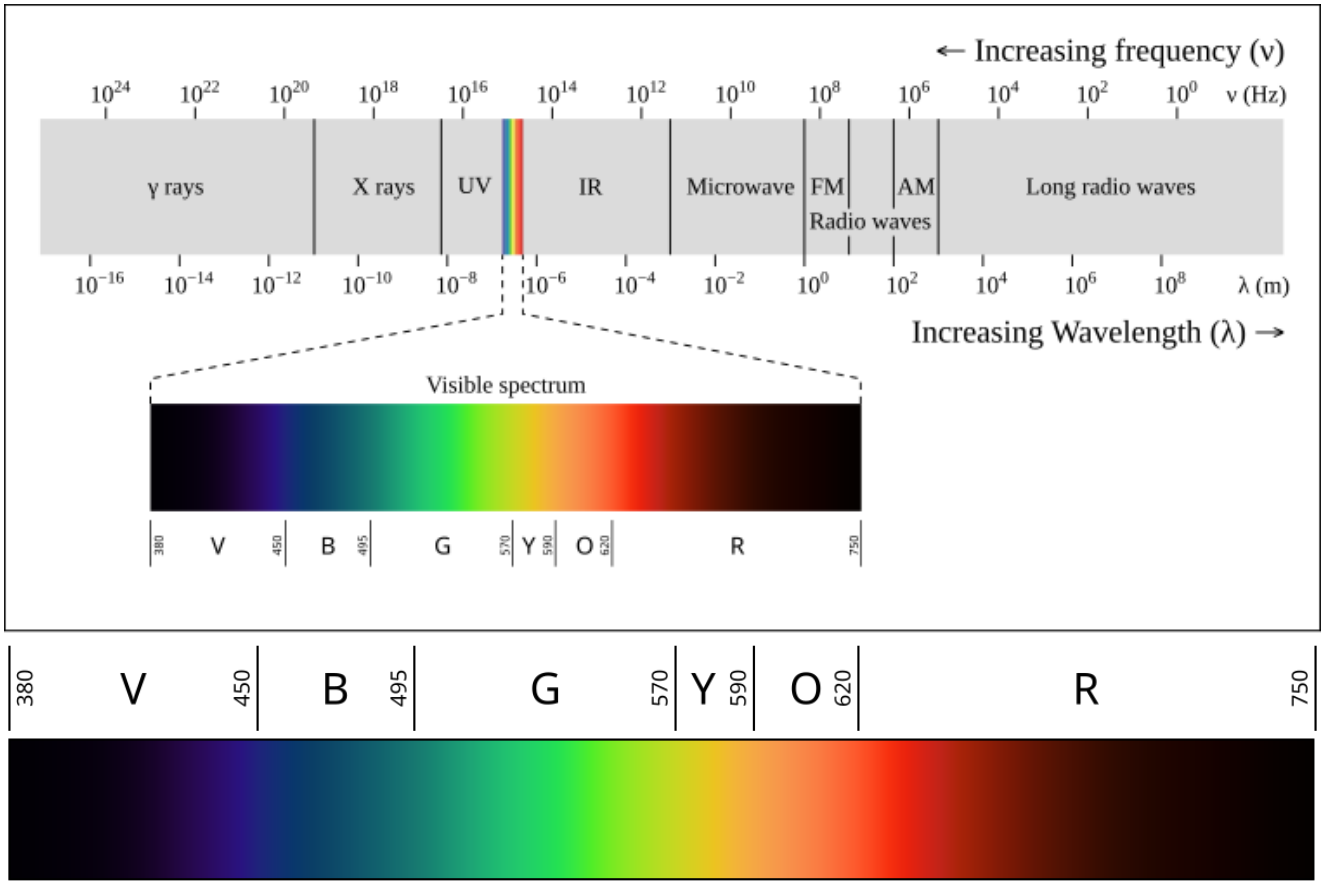
sunt programe care rulează pe GPU (placa video).

Shaderele sunt compilate din cod sursă scrise în limbajul GLSL (sau altul similar).

Lumina vizibilă

Ce este lumina?

Radiația electromagnetică vizibilă ochiului uman!

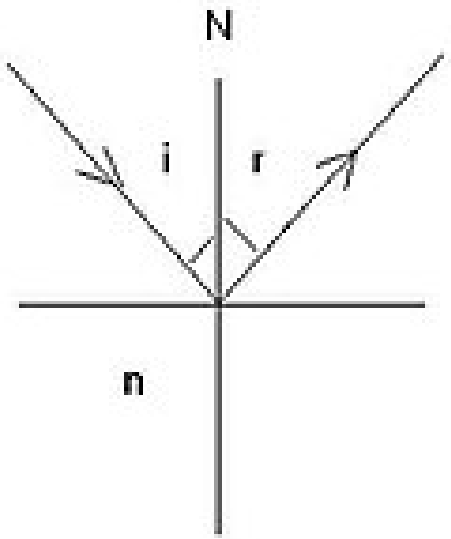


Reflexia luminii

Sunt două legi:

1. Raza de lumină incidentă, raza de lumină reflectată și normala la suprafață sunt coplanare în punctul de incidență.
2. Unghiul de reflexie $\hat{r}$ este egal cu unghiul de incidență $\hat{i}$. Matematic, a doua lege se scrie

$$\hat{i} = \hat{r}$$

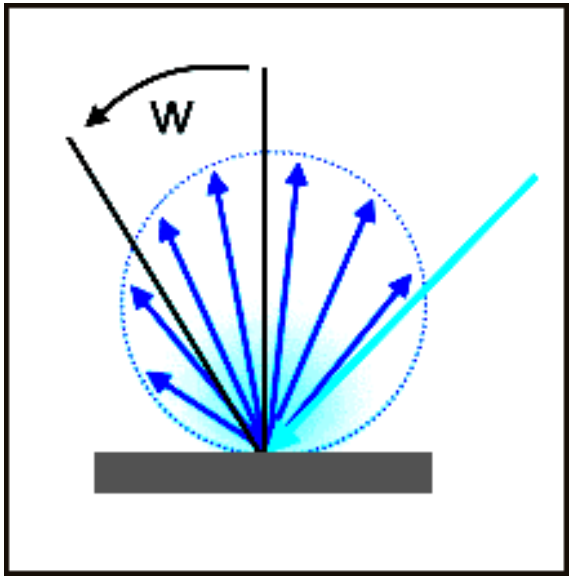


Legea lui Lambert

Reflexia este cu atât mai intensă cu cât suprafața pe care este reflectată este mai netedă:

$$I = S \cdot cos(\omega) \cdot B$$

I - intensitatea luminii reflectate.
S - coeficientul de reflecție a suprafeței(depinde de material). $cos(\omega)$ - unghiul de care depinde reflecția (legea cosinusului lui Lambert).
B - intensitatea luminii incidente.



Refracția

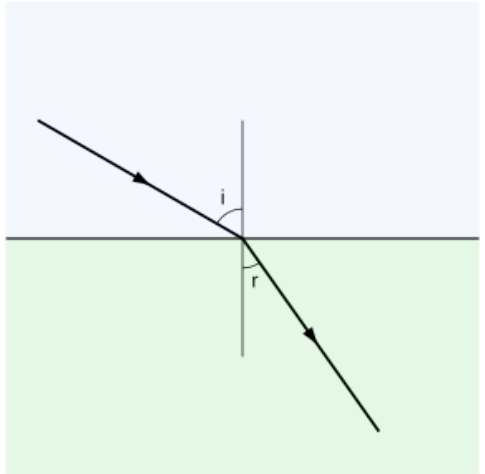
Schimbarea direcției de propagare a unei unde la interfața dintre două medii în care unda are viteze de propagare diferite, sau datorită gradientului local al proprietăților mediului în care se propagă.

Apare când lumina trece dintr-un mediu transparent (aer, apă, sticlă etc.) în altul.

Legile Refracției

1. Raza incidentă, normala (în punctul de incidență) și raza refractată sunt coplanare.
2. Indicele de refracție notat cu **n** este raportul dintre sinusul unghiului de incidență $\hat{i}$ și sinusul unghiului de refracție $\hat{r}$.

$$n = \frac{sin\hat{i}}{sin\hat{r}}$$



Culoarea Luminii

Percepția de către ochi a uneia sau a mai multor frecvențe (sau lungimi de undă) de lumină.

Caracterizarea culorii corpurilor - o suprafață care reflectă lumina reflectă independent fiecare componentă spectrală (fiecare frecvență sau, echivalent, fiecare lungime de undă).

Dacă radiația incidentă are distribuția spectrală a puterii $s(\lambda)$ și suprafața are funcția coeficient de reflexie pe lungime de undă $c(\lambda)$, lumina reflectată va avea distribuția spectrală a puterii dată de $r(\lambda) = s(\lambda) \cdot c(\lambda)$.

$$reflectedLight = LightColor * SurfaceColor$$

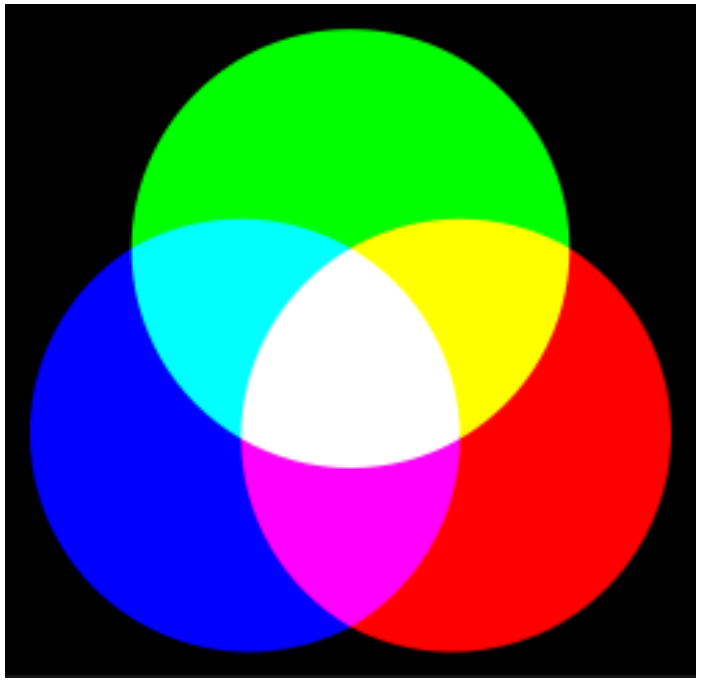
Culoarea Luminii

```
GLfloat lightColor[] = {1.0f, 1.0f, 1.0f};
GLfloat surfaceColor[] = {0.9f, 0.1f, 0.1f};
GLfloat reflectedLight[] = {
lightColor[0] * surfaceColor[0], // R = 1.0 * 0.9 = 0.9
lightColor[1] * surfaceColor[1], // G = 1.0 * 0.1 = 0.1
lightColor[2] * surfaceColor[2]  // B = 1.0 * 0.1 = 0.1
 };
GLfloat reflectedLight[] = {0.9f, 0.1f, 0.1f};
```

Sinteza culorilor

Sinteza aditivă

Lumina de o anumită culoare se obține prin combinarea unor surse de lumină de diferite culori fixate, numite culori *primare*.



Sinteza culorilor

Sinteza substractivă

C

uloarea se obține prin filtrări succesive ale unei surse albe prin filtre de diferite culori și *tării* ale filtrării.

Modelarea luminii

- în realitate lumina e extrem de complicată – aproape imposibil de simulat perfect
- folosim aproximații pe modele simplificate ce permit procesarea și arată similar cu realitatea
- Categorii de modele:
  1. **Modele globale** – ex: ray tracing, lumen (Unreal5)
     extrem de costisitoare computațional, rezultate spectaculoase
  2. **Modele locale** – ex: modelul Phong
     "ieftine" computațional, rezultate adecvate

Surse de lumină

Putem avea două tipuri de surse de lumină

- punctuale – surse de lumină locale
- direcționale – surse de lumină aflate la infinit

Matricea Model-view-projection afectează lumina

Diferite efecte apar când sunt specificate:

- coordonatele camerei,
- coordonatele globale,
- coordonatele modelului.

Lumina punctuală

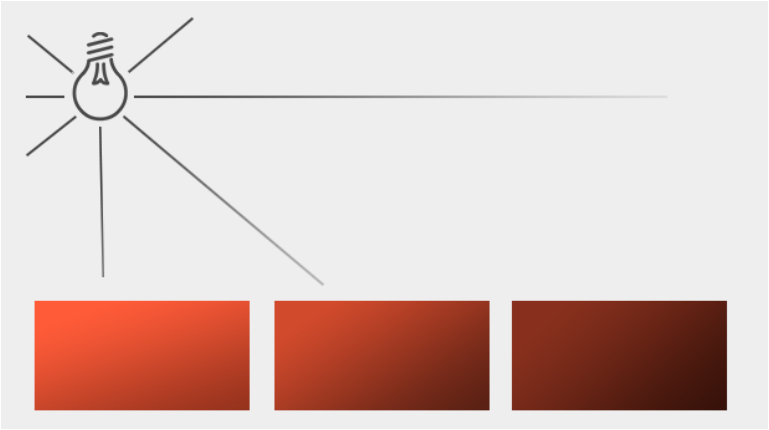


Figure: Lumina punctuală (spot light)

Lumina direcțională

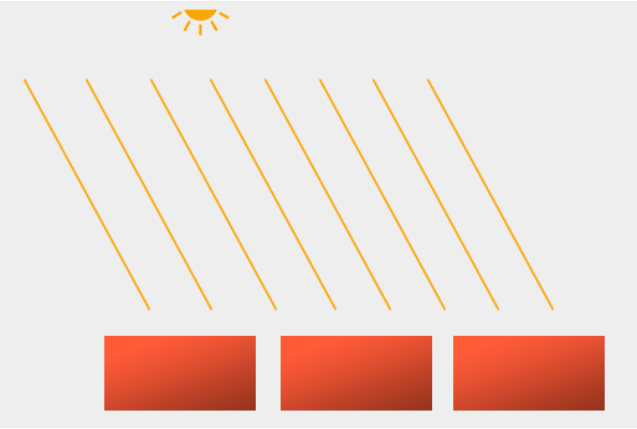


Figure: Lumina direcțională (directional Light)

Modele locale

Se numesc modele locale pentru că nu folosesc pentru a calcula luminozitatea unui punct alte informații decât cele locale (poziție, normală, tangentă, textură, etc.) și cele legate de sursele de lumină (poziții, tipuri, etc.)

Aventaje:

- folosesc doar informație locală;
- nu consumă multe resurse.

Dezavantaje:

- nu țin cont de obiectele din jur – dificil de realizat direct: transparența, reflecția, refracția și umbrele.

Modelul Phong

Dezvoltat de *Bui Tuong Phong* la universitatea din Utah (1973) și extins de *James F. Blinn* (1977).

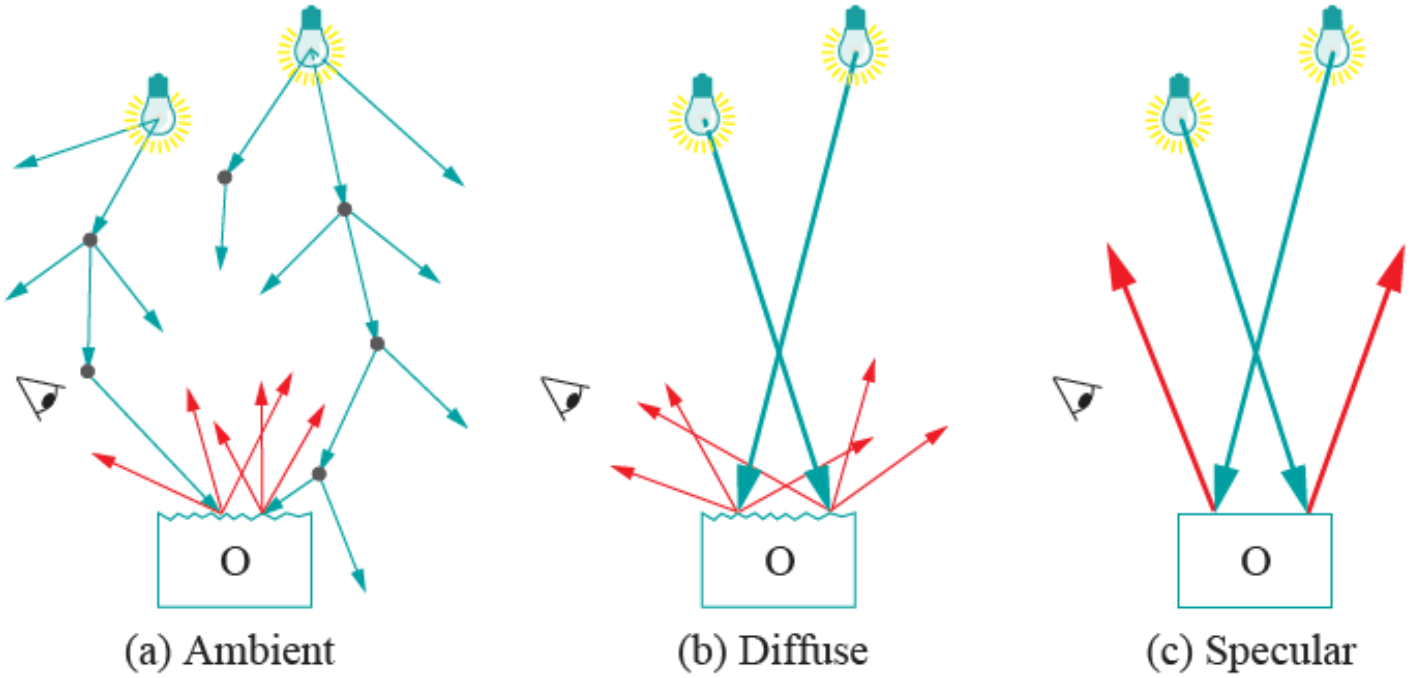


Figure: Cele trei componente ale modelului lui Phong.

Modelul Phong

Este o combinație de trei componente principale:

- *Ambient lighting* – lumină ambientală

  chiar și pe întuneric vedem ceva (întunericul nu este absolut), undeva există o sursă de lumină. Simulăm acest lucru folosind o constantă care dă întotdeauna ceva culoare obiectelor.

- *Diffuse lighting* – lumină difuză

  simulează impactul ce îl are lumina asupra unui obiect ce se găsește pe direcția ei. E cea mai importantă componentă a acestui model. Practic cu cât o parte a unui obiect se găsește în dreptul luminii cu atât e mai luminos.

- *Specular lighting* – lumina speculară

  simulează acel punct luminos care apare pe obiectele sclipitoare. Aceste accente au mai mult culoarea luminii decât culoarea obiectului.

Culoarea luminii

Modelul Phong

va fi modelată compunând-o din 4 componente:

- **culoarea difuză** (diffuse color) – culoarea luminii reflectată difuz de material
- **culoarea speculară** (specular color) – culoarea luminii reflectată specular de material
- **culoarea ambientală** (ambient color) – culoarea luminii ambientală reflectată de material
- **culoarea emisă** (emission color) – nu e o relfecție efectivă a luminii ci se referă la capacitatea materialului de a emite o lumină de o anumită culoare (deobicei este neagră $(0.0, 0.0, 0.0)$).

Ambient Lighting

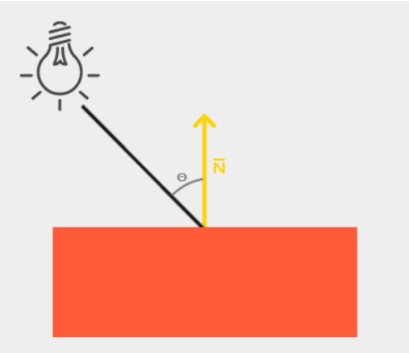
- simulează lumina slabă care se gasește peste tot
- se pune în fragment shader:

```
// ambient
float ambientStrength = 0.2;
vec3 ambient = ambientStrength * lightColor;

    ...

FragColor = vec4(ambient + diffuse + specular, 1.0) * aColor;
```

Diffuse Lighting



- în din vertex shader se transmit normalele în fiecare nod
- în fragment shader se folosesc aceste normale:

```
// diffuse
vec3 norm = normalize(nCoord);
vec3 lightDir = normalize(lumina - bPos);
float diff = max(dot(norm, lightDir), 0.0);
vec3 diffuse = diff * lightColor;
    ...
FragColor = vec4(ambient + diffuse + specular, 1.0) * aColor;
```

Diffuse Lighting

- lumina – coordonatele luminii, bPos coordonatele nodurilor, nCoord normalelor
- normalele într-un nod A la suprafața triunghiulară ABC pot fi determinate prin produsul vectorial

$$BA \times CA$$

Atenție la direcția normalei (trebuie sa fie spre exteriorul obiectului)!!

Normalele la un punct

Probleme

În urma transformărilor efectuate de matricea *model* normalele transmise în vertex shader **nu mai corespund** normalelor reale.

Soluție: transformarea normalelor în mod similar cu cea a vertexurilor

```
gl_Position = projection*view*model*vec4(aPos, 1);
nCoord = mat3(transpose(inverse(model))) * normale;
```

practic transformăm normalele cu transpusa inversei matricii *model*.

Specular Lighting

- în fragment shader se transmite pozitia camerei și se adaugă componenta speculară

```
// specular
float specularStrength = 0.5;
vec3 viewDir = normalize(viewPos - bPos);
vec3 reflectDir = reflect(-lightDir, norm);
float spec = pow(max(dot(viewDir, reflectDir), 0.0), 32);
vec3 specular = specularStrength * spec * lightColor;
```

Modele Globale

Avantaje:

- Realism - simulează comportamentul real al luminii
- Iluminare indirectă și umbre mai naturale
- Fenomenul de bouncing light (reflexia luminii)
- Simularea efectului de ambient occlusion
- Interacțiune mai bună între materiale și lumina ambientală

Modele Globale

Dezavantaje:

- **Cost computațional ridicat** – Modelele de iluminare globală necesită resurse hardware semnificative, mai ales în cazul randărilor în timp real.
- **Necesită optimizări** – Tehnici precum "baking" sau upscaling sunt adesea utilizate pentru a reduce timpul de calcul.

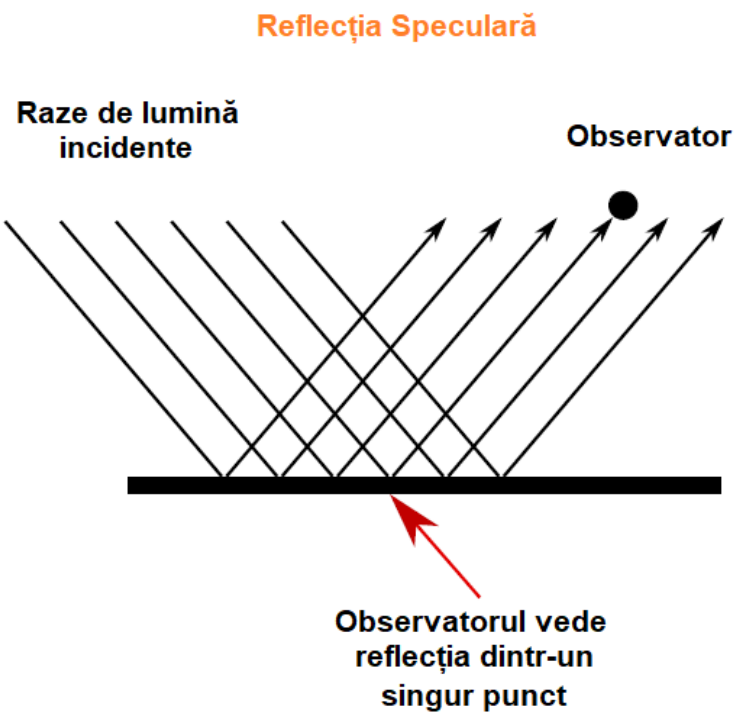
Materiale și Lumină

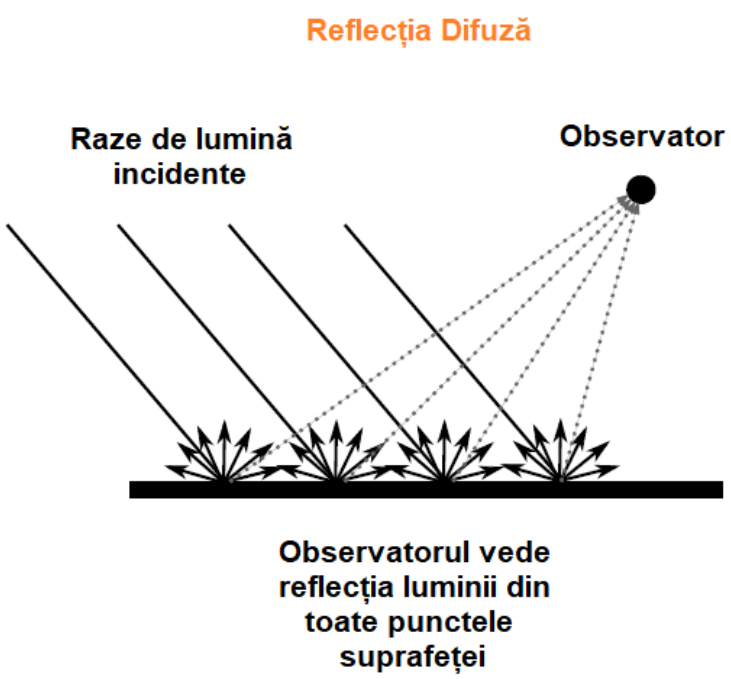
Orice obiect real este constituit dintr-un material:

- lemn brut– se văd fibrele, este rugos, nu strălucește, ...
- sticlă – este lucioasă, transparentă, netedă, ...
- metal – are luciu specific, ...
- ceramică – este netedă, stălucire medie (în funcție de finisaj), ...
- etc.

Fiecare material are proprietățiile lui care pentru o imagine cât mai realistă trebuie luată în considerare.

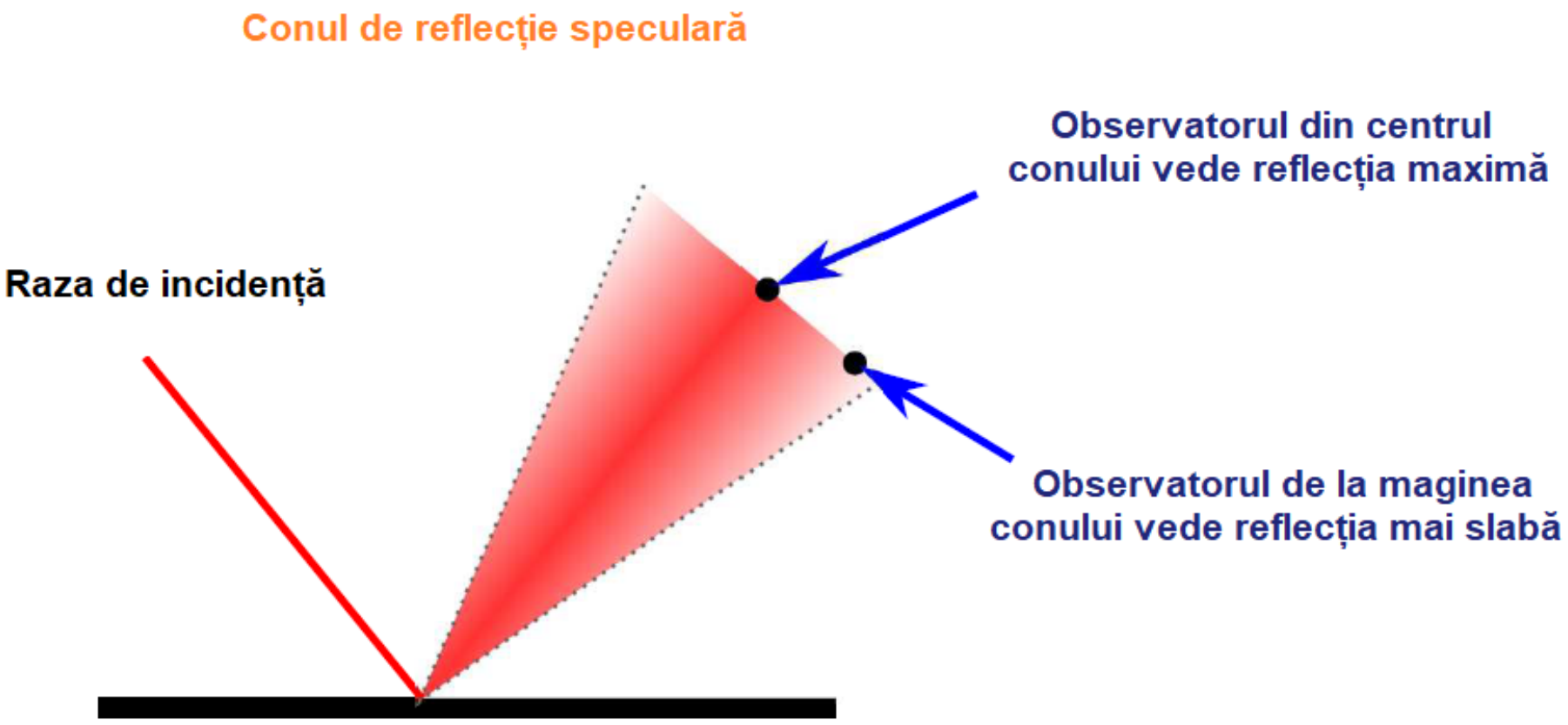
Reflecția luminii





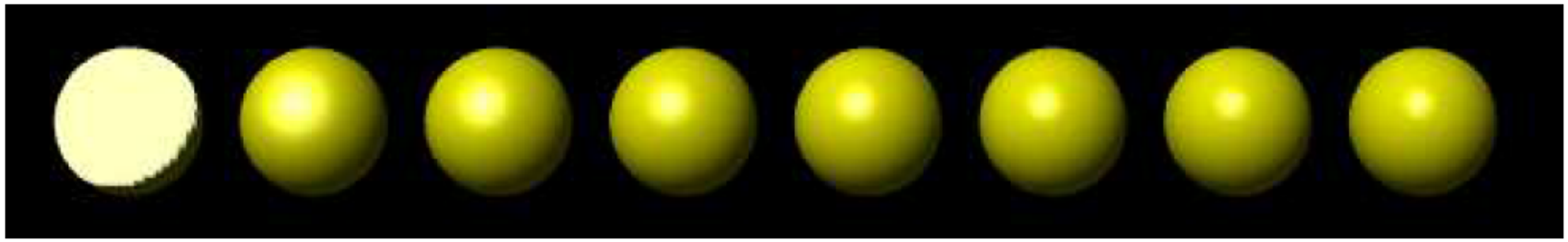
Conul de reflecție

Modelarea accentelor speculare este făcută practic cu un con:



dacă materialul este mai lucios conul este mai îngust, dacă materialul este mai mat conul este mai larg.

Strălucirea



- plecând de la stânga (strălucire 0) – o imagine nereușită
- mergând spre dreapta de la materiale mate spre strălucitoare
- ultimul element de la dreapta (strălucire maximă) – componenta speculară e aproape un punct

Ecuația Luminii

$$I = I_a k_a + I_d k_d (\vec{N} \cdot \vec{L}) + I_s k_s (\vec{R} \cdot \vec{V})^n$$

unde:

- I este intensitatea totală a luminii percepută.
- I_a, I_d, I_s reprezintă intensitățile surselor de lumină ambientală, difuză și specular, respectiv.
- k_a, k_d, k_s sunt coeficienții materialului pentru reflectanța ambientală, difuză și specular, respectiv.
- $\vec{N}$ este vectorul normal la suprafață.
- $\vec{L}$ este vectorul care indică direcția de la suprafață către sursa de lumină.
- $\vec{R}$ este vectorul de reflexie al luminii pe suprafață.
- $\vec{V}$ este vectorul care indică direcția de la suprafață către punctul de vedere al observatorului.
- n este exponentul de luciu, care determină "strălucirea" reflexiei.

Modelarea materialului

În practică proprietățile materialului de a reacționa la lumină (culoarea materialului) sunt date de 4 parametri:

- ambient – (vector de 3 elemente float)
- diffuse – (vector de 3 elemente float)
- specular – (vector de 3 elemente float)
- shininess – (float)

Cu ajutorul acestor culori putem face materialului un 'fine tunning'

Exemple de materiale

Name	Ambient	Diffuse
emerald	(0.02, 0.17, 0.02)	(0.07, 0.61, 0.07)
jade	(0.13, 0.22, 0.15)	(0.54, 0.89, 0.63)
obsidian	(0.05, 0.05, 0.06)	(0.18, 0.17, 0.22)
pearl	(0.25, 0.20, 0.20)	(1.00, 0.82, 0.82)
ruby	(0.17, 0.01, 0.01)	(0.61, 0.04, 0.04)

Name	Specular	Shininess
emerald	(0.63, 0.72, 0.63)	0.6
jade	(0.31, 0.31, 0.31)	0.1
obsidian	(0.33, 0.32, 0.34)	0.3
pearl	(0.29, 0.29, 0.29)	0.088
ruby	(0.72, 0.62, 0.62)	0.6

Table: culorile și strălucirea câtorva materiale pe componente

Exemple de materiale

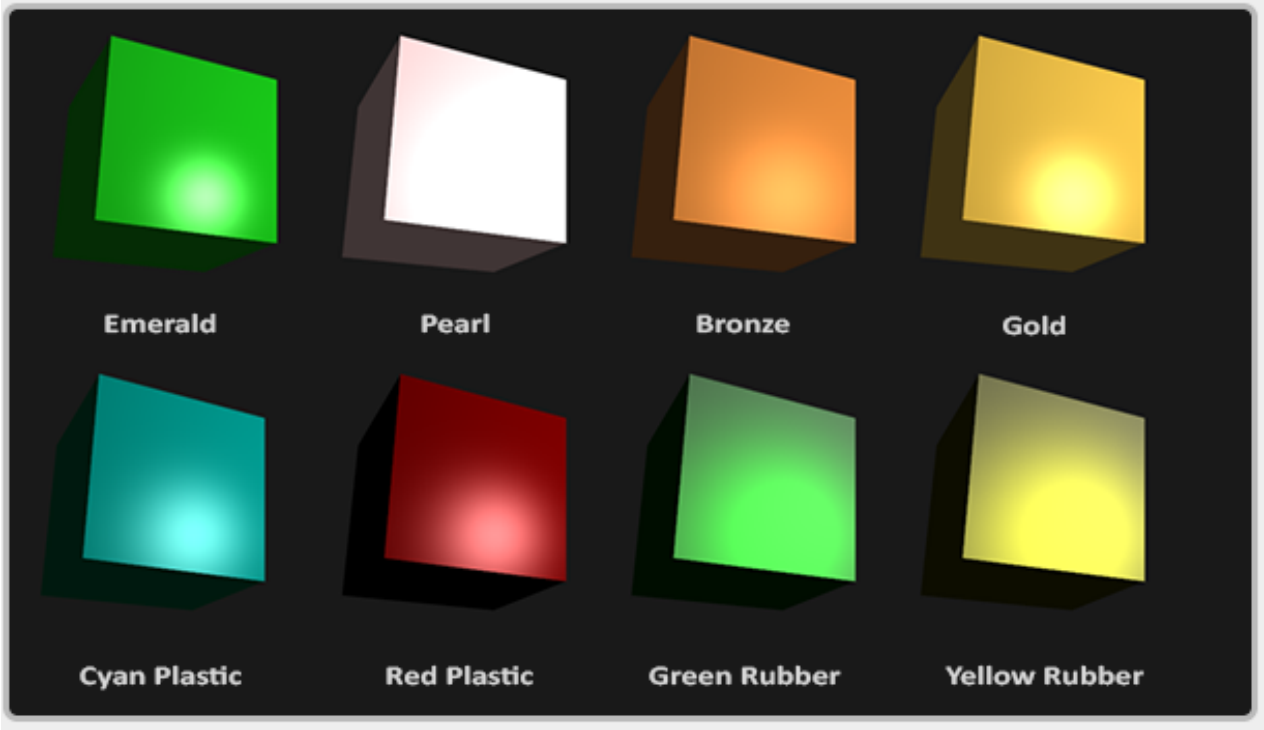


Figure: Câteva materiale. Primele 5 sunt construite conform tabelului anterior

Exemplu de cod

În fragment shader se definește structura Material și se inițializează o variabilă uniformă de acest tip:

```
#version 330 core
struct Material {
    vec3 ambient;
    vec3 diffuse;
    vec3 specular;
    float shininess;
};

uniform Material material;
```

Exemplu de cod

Avem apoi în funcția main din fragment shader:

```
void main()
{
    // ambient
    vec3 ambient = lightColor * material.ambient;
    // diffuse
    vec3 norm = normalize(Normal);
    vec3 lightDir = normalize(lightPos - FragPos);
    float diff = max(dot(norm, lightDir), 0.0);
    vec3 diffuse = lightColor * (diff * material.diffuse);
    // specular
    vec3 viewDir = normalize(viewPos - FragPos);
    vec3 reflectDir = reflect(-lightDir, norm);
    float spec = pow(max(dot(viewDir, reflectDir), 0.0), mater
    vec3 specular = lightColor * (spec * material.specular);
```

Exemplu de cod

```
    vec3 result = ambient + diffuse + specular;
    FragColor = vec4(result, 1.0);
}
```

Pentru a transmite din programul nostru C++ valorile vom determina în mod normal locațiile ca la orice variabilă uniformă. De exemplu:

```
locatie = glGetUniformLocation(programul_nostru,
                                        "material.ambient");
```

După care se inițializează ca orice altă variabilă uniformă.

Bibliografie

www.scratchapixel.com
www.opengl.org
learnopengl.com